ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA KỸ THUẬT MÁY TÍNH



ĐỒ ÁN MÔN HỌC

THIẾT KẾ HỆ THỐNG NHÚNG KHÔNG DÂY

Môn học: Thiết kế hệ thống nhúng không dây - CE232.Q21

Giảng viên hướng dẫn: TS. Đoàn Duy

Thực hiện bởi:

Mã số sinh viên	Họ và Tên
23521806	Huỳnh Tấn Vũ
23521798	Trần Danh Vinh

TP. HỒ CHÍ MINH, 2026

MỤC LỤC

Chương 1. TỔNG QUAN 1
1.1. Tính cấp thiết và lý do chọn đề tài 2
1.2. Mục tiêu nghiên cứu 3
1.3. Đối tượng và phạm vi nghiên cứu 3
1.3.1. Đối tượng nghiên cứu 3
1.3.2. Phạm vi nghiên cứu 4
Chương 2. CƠ SỞ LÝ THUYẾT 5
2.1. Cơ sở lý thuyết về điều khiển tự động 5
2.1.1. Hệ thống điều khiển vòng kín. 5
2.1.2. Thuật toán điều khiển PID 5
2.1.3. Rời rạc hóa thuật toán điều khiển PID cho vi điều khiển 6
2.2. Hệ điều hành thời gian thực 7
2.2.1. Tổng quan về FreeRTOS 7
2.2.1.1. Khái niệm hệ điều hành thời gian thực (RTOS) 7
2.2.1.2. Lý do chọn FreeRTOS cho ESP32 8
2.2.1.3. Kiến trúc nhân (Kernel Architecture) của FreeRTOS. 8
2.2.1.4. Nguyên lý khởi chạy của hệ thống FreeRTOS 8
2.2.2. Quản lý đa tác vụ 9
2.2.2.1. Các trạng thái của một tác vụ 9
2.2.2.2. Bộ lập lịch phân phối theo độ ưu tiên 9
2.2.2.3. Quản lý đa lõi đối xứng trên ESP32 9
2.2.3. Cơ chế giao tiếp nội bộ 9
2.2.3.1. Khái niệm về giao tiếp nội bộ trong RTOS 9
2.2.3.2. Giải pháp chia sẽ dữ liệu trong đồ án 10
2.3. Giao thức truyền thông & mạng 11
2.3.1. Giao thức MQTT 11
2.3.1.1. Khái niệm về giao thức MQTT 11
2.3.1.2. Mô hình kiến trúc Publish/Subscribe 11
2.3.1.3. Các đặc tính kỹ thuật cốt lõi ứng dụng trong đồ án 11
2.3.1.4. Lý do lựa chọn MQTT cho đồ án 12
2.3.2. Hạ tầng mạng ảo (Mesh VPN) & NAT Traversal 12
2.3.2.1. Rào cản từ mạng nội bộ và cơ chế NAT 12
2.3.2.2. Kỹ thuật đục lỗ NAT (NAT Traversal) 12
2.3.2.3. Giải pháp mạng ảo dạng lưới (Mesh VPN) qua Tailscale 12
2.4. Tổng quan về các chuẩn giao tiếp phần cứng 13

2.4.1. Giao tiếp I2C 13
2.4.1.1. Khái niệm và Kiến trúc vật lý của I2C 13
2.4.1.2. Nguyên lý truyền nhận dữ liệu cơ bản 13
2.4.1.3. Ứng dụng và tính năng nâng cao trong đồ án (SHT31) 14
2.4.2. Kỹ thuật băm xung (PWM) 14
2.4.2.1. Tổng quan và khái niệm PWM 14
2.4.2.2. Ứng dụng cho hệ thống của nhóm 15
2.5. Linh kiện cốt lõi sử dụng 15
2.5.1. Vi điều khiển ESP32-S3 - MCU 15
2.5.2. Cảm biến SHT31 - Sensor 16
2.5.3. Sò Peltier (TEC) - Actuator 16
2.5.4. Mạch cầu H DRV8833 17
2.5.5. Raspberry Pi 4 17
Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 19
3.1. Phân tích yêu cầu hệ thống 19
3.1.1. Yêu cầu chức năng 19
3.1.2. Yêu cầu phi chức năng 19
3.2. Sơ đồ khối tổng thể của hệ thống 21
3.2.1. Sơ đồ kiến trúc phân tầng hệ thống 21
3.2.2. Sơ đồ kết nối phần cứng 23
3.2.3. Chức năng chi tiết các khối trong hệ thống 23
3.2.4. Bản chất luồng tín hiệu và truyền thông giữa các khối 24
3.3. Thiết kế chi tiết phần cứng 24
3.3.1. Cảm biến SHT31 25
3.3.2. Mạch cầu H DRV8822 và sò Peltier 25
3.3.3. Nguyên tắt chung mass và nguồn cấp 25
3.4. Thiết kế chi tiết phần mềm 25
3.4.1. Sơ đồ kiến trúc phần mềm và phân bổ tác vụ 25
3.4.2. Lưu đồ thuật toán điều khiển chính 26
3.4.3. Luồng hoạt động chính của hệ thống 26
3.4.4. Luồng hoạt động và các cơ chế tối ưu thuật toán PID trong hệ thống 29
3.4.5. Cơ chế bảo vệ phần cứng và bộ lọc dữ liệu đám mây 30
3.5. Thiết kế truyền thông và giao thức 30
3.5.1. Kiến trúc mạng và mô hình truyền thông MQTT 30
3.5.2. Quy hoạch cấu trúc Topic mạng 30
3.5.3. Định dạng cấu trúc bản tin dữ liệu (Payload Format) 31

3.5.3.1. Bản tin giám sát 31
3.5.3.2. Bản tin điều khiển Setpoint 31
3.5.4. Giải pháp bảo mật và định tuyến qua Mesh VPN 31
Chương 4. KẾT QUẢ THỰC NGHIỆM 32
4.1. Kết quả thi công phần cứng 32
4.2. Kết quả truyền thông mạng và giao diện người dùng 33
4.2.1. Kết quả khởi tạo kết nối và truyền tin tại thiết bị biên. 33
4.2.2. Kết quả xuyên thấu tường lửa bằng mạng riêng ảo Tailscale. 34
4.2.3. Kết quả thiết kế giao diện giám sát và điều khiển. 35
4.3. Đánh giá đáp ứng của thuật toán PID 36
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 38
5.1. Kết quả đạt được 38
5.2. Phần hạn chế của đề tài 38
5.3. Hướng phát triển 38

DANH MỤC HÌNH VẼ

Hình 1.1: Sơ đồ chia sẻ dữ liệu giữa các task..... 11
Hình 1.2: Sơ đồ kiến trúc phân tầng hệ thống.....21
Hình 1.3: Sơ đồ kết nối phần cứng tổng thể.....23
Hình 1.4: Schematic thiết kế phần cứng..... 25
Hình 1.5: Sequence Diagram của hệ thống.....27
Hình 1.6: Sơ đồ thuật toán PID.....29
Hình 1.7: Kết quả lắp ghép phần cứng.....32
Hình 1.8: Sơ đồ nguyên lí hoạt động khối chấp hành.....33
Hình 1.9: Giao diện giám sát và điều khiển.....35

BẢNG PHÂN CHIA NỘI DUNG CÔNG VIỆC

Mã số sinh viên	Họ và Tên	Nội dung công việc
23521806	Huỳnh Tấn Vũ	● Thực hiện giao tiếp I2C với cảm biến. ● Áp dụng giao thức MQTT vào hệ thống. ● Xây dựng gateway và giao diện giám sát, điều khiển (NODE-RED) trên RaspBerry Pi 4.
23521798	Trần Danh Vinh	● Thiết kế, lắp đặt linh kiện phần cứng. ● Triển khai thuật toán PID cho các động cơ chấp hành.

Chương 1. **TỔNG QUAN**

1.1. Tính cấp thiết và lý do chọn đề tài

Trong kỷ nguyên vạn vật kết nối (IoT), nhu cầu giám sát và điều khiển nhiệt độ tự động không chỉ giới hạn trong các dây chuyền công nghiệp quy mô lớn mà đã mở rộng mạnh mẽ sang môi trường dân dụng, không gian làm việc và các phòng thí nghiệm vi mô. Việc duy trì một môi trường nhiệt độ ổn định với độ chính xác cao là yếu tố sống còn để đảm bảo chất lượng cho nhiều quy trình lưu trữ và thử nghiệm vật lý.

Từ những rào cản thực tiễn trên, việc nghiên cứu và phát triển một hệ thống điều khiển nhiệt độ thông minh, vừa đảm bảo tính đáp ứng thời gian thực tại biên, vừa có khả năng quản trị từ xa an toàn mà không cần giao diện cơ học là một bài toán kỹ thuật hết sức cấp thiết. Đó cũng chính là lý để nhóm quyết định lựa chọn đề tài: "Thiết kế hệ thống giám sát và điều khiển nhiệt độ IoT ứng dụng hệ điều hành thời gian thực và mạng riêng ảo".

Đề tài hướng tới việc xây dựng một thiết bị biên (Edge Device) theo kiến trúc Headless, tập trung giải quyết bài toán thông qua ba trụ cột công nghệ cốt lõi của ngành Kỹ thuật Máy tính:

- Tối ưu hóa Điều khiển thời gian thực: Ứng dụng nhân hệ điều hành FreeRTOS trên vi điều khiển ESP32 để cấp phát tài nguyên độc lập cho thuật toán PID. Điều này giúp cơ cấu chấp hành (sò nhiệt điện Peltier) luôn hoạt động bám sát giá trị cài đặt với sai số và độ trễ thấp nhất mà không bị gián đoạn bởi các tác vụ mạng.
- Truyền thông IoT tinh gọn: Sử dụng mô hình Publish/Subscribe của giao thức MQTT để trao đổi liên tục các bản tin giám sát môi trường dạng chuỗi JSON, đảm bảo tính liên tục của dữ liệu mà không gây nghẽn băng thông.
- Giải pháp Xuyên thấu mạng và Bảo mật: Triển khai hạ tầng mạng ảo dạng lưới (Mesh VPN) thông qua nền tảng Tailscale (dựa trên chuẩn mã hóa

WireGuard). Giải pháp này giúp thiết bị tự động đục lỗ NAT thành công, cho phép người dùng giám sát và cấu hình hệ thống từ bất kỳ đâu qua Internet và tăng tính bảo mật.

Việc thực hiện đề tài này không chỉ giúp tổng hợp và vận dụng linh hoạt các kiến thức về kiến trúc vi xử lý, hệ điều hành thời gian thực và an toàn mạng, mà còn tạo ra một sản phẩm nguyên mẫu hoàn chỉnh, bắt kịp xu hướng phát triển kiến trúc Cloud-centric của các hệ thống nhúng hiện đại.

1.2. Mục tiêu nghiên cứu

- Đề tài hướng tới việc điều khiển nhiệt độ trong không gian hẹp, kết hợp với xu hướng giám sát phi tập trung của IoT.
- Xây dựng thành công hệ thống điều khiển vòng kín áp dụng thuật toán điều khiển tỉ lệ - tích phân - đạo hàm (PID) rời rạc.
- Thiết kế hệ thống phần cứng điều khiển, tối ưu hóa tài nguyên vi điều khiển ESP32-S3.
- Ứng dụng hệ điều hành thời gian thực FreeRTOS để quản lý đa nhiệm, tách biệt luồng xử lý phần cứng và luồng truyền thông mạng. Xây dựng máy chủ trung tâm (IoT Gateway) trên nền tảng Raspberry Pi, sử dụng giao thức MQTT và Node-RED để vẽ biểu đồ giám sát trực quan. Thiết lập mạng riêng ảo (Mesh VPN) qua Tailscale nhằm vượt rào cản tường lửa mạng diện rộng, cho phép điều khiển và cấu hình hệ thống từ xa một cách bảo mật.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

- Đặc tính nhiệt điện, cấu trúc bán dẫn và quán tính nhiệt của sò nóng lạnh Peltier, cùng nguyên lý đảo chiều dòng điện của IC công suất DRV8833.
- Phương pháp rời rạc hóa phương trình PID.

- Kỹ thuật điều chế độ rộng xung (PWM) tần số cao và cơ chế lập lịch tác vụ (Task Scheduling), cấp phát bộ nhớ dùng chung (Shared Memory) của lõi FreeRTOS.
- Giao thức truyền tin gọn nhẹ MQTT, cấu trúc dữ liệu JSON, nền tảng SCADA mã nguồn mở Node-RED và công nghệ mạng Zero-Trust WireGuard (Tailscale).

1.3.2. Phạm vi nghiên cứu

- Đề tài chỉ tập trung đo lường và điều khiển nhiệt độ. Mặc dù cảm biến SHT31 có hỗ trợ đo độ ẩm nhưng tính năng này được nhóm chủ động loại bỏ khỏi khối mã nguồn để tối ưu hóa thời gian xử lý bus I2C.
- Không gian làm việc là một hộp cách nhiệt thể tích nhỏ, dải nhiệt độ điều khiển xoay quanh mức nhiệt độ phòng trong khoảng từ 15°C đến 24°C.
- Sử dụng vi điều khiển ESP32-S3, Module SHT31, IC DRV8833 và tải là 01 sò Peltier 5V DC đi kèm hệ thống tản nhiệt khí.
- Hệ thống giám sát (UI/UX) chỉ được triển khai trên nền tảng web thông qua Node-RED tại máy chủ cục bộ Raspberry Pi. Đề tài không đi sâu vào việc phát triển ứng dụng di động riêng biệt hay thuê máy chủ cloud từ các nhà cung cấp bên thứ ba như AWS, Azure,... để tiết kiệm chi phí vận hành.

Chương 2. **CƠ SỞ LÝ THUYẾT**

2.1. Cơ sở lý thuyết về điều khiển tự động

2.1.1. Hệ thống điều khiển vòng kín.

- Trong kỹ thuật điều khiển tự động, các hệ thống thường được chia thành hai loại chính đó là vòng hở và vòng kín. Trong phạm vi đề tài này, để đảm bảo nhiệt độ của hộp sò Peltier luôn bám sát với giá trị setpoint mà không bị ảnh hưởng bởi các nhiễu loạn từ môi trường bên ngoài, nhóm quyết định xây dựng hệ thống theo mô hình điều khiển vòng kín.
- Đặc trưng cơ bản của hệ thống vòng kín là có sự xuất hiện của tín hiệu phản hồi (Feedback). Cụ thể là cảm biến SHT31 sẽ liên tục đo nhiệt độ thực tế của môi trường và gửi về vi điều khiển, sau đó hệ thống sẽ tính toán sai lệch (Error) giữa nhiệt độ mong muốn và nhiệt độ thực tế theo công thức sau:

$$\mathbf{e(t) = Setpoint - Temp_{thựctế}(t)}$$

- Dựa vào giá trị e(t) tính được này, bộ điều khiển trung tâm sẽ quyết định thuật toán bù trừ và xuất tín hiệu điều khiển (PWM) ra mạch công suất để băm xung cho sò Peltier, ép gía trị Error e(t) này tiến dần về 0.

2.1.2. Thuật toán điều khiển PID

- Để giải quyết bài toán nhiệt của khối nhôm và sò Peltier, nhóm ứng dụng thuật toán điều khiển PID. Đây là bộ điều khiển phản hồi thông dụng và hiệu quả được sử dụng trong nhiều lĩnh vực khác nhau, bao gồm 3 phần tác động: Tỉ lệ (Proportion - P), Tích phân (Integral - I) và Đạo hàm (Derivative - D).
- Phương trình toán học của bộ điều khiển PID trong miền thời gian liên tục được biểu diễn như sau:

$$u(t) = K_p * e(t) + K_i \int_0^t e(\tau)d\tau + K_d * \frac{de(t)}{dt}$$

Với u(t): Tín hiệu đầu ra của bộ điều khiển PID

e(t): Sai số hiện tại.

K_p, K_i, K_d: Lần lượt là các hệ số khuếch đại của P, I, D.

- **Tỉ lệ (Proportion - P):** $u_p(t) = K_p * e(t)$

Tạo ra tín hiệu điều khiển tỉ lệ thuận với sai số hiện tại. PWM càng cao khi nhiệt độ càng xa đích. Tuy nhiên, nếu chỉ dùng riêng khâu P, hệ thống sẽ luôn tồn tại sai số xác lập tức là không bao giờ chạm đúng vạch Setpoint.

- **Tích phân (Intergral - I):** $u_I(t) = K_i \int_0^t e(\tau)dt$

Bước này sẽ cộng dồn các sai số trong quá khứ. Tích phân I đóng vai trò triệt tiêu hoàn toàn sai số xác lập, giúp hệ thống điều chỉnh để nhiệt độ thực tế bằng chính xác với nhiệt độ setpoint.

- **Đạo hàm (Derivative - D):** $u_D(t) = K_d * \frac{de(t)}{dt}$

Tiếp đến là dự đoán xu hướng thay đổi của sai số trong tương lai dựa trên tốc độ biến thiên hiện tại. Đạo hàm D giúp hệ thống không bị vượt lố khi nhiệt độ đang lao nhanh về sát setpoint.

2.1.3. Rời rạc hóa thuật toán điều khiển PID cho vi điều khiển

- Do vi điều khiển không thể xử lý được các phép toán tích phân và đạo hàm liên tục, nên nhóm tiến hành rời rạc hóa phương trình PID theo chu kỳ lấy mẫu Δt với chu kỳ vòng lặp hệ thống là 1s.
- Để thuật toán hoạt động ổn định trong thực tế, nhóm đã nghiên cứu và lập trình thêm các cơ chế bảo vệ nâng cao cho bộ PID. Bằng cách sử dụng dạng rời rạc của phương trình PID có dạng như sau:

$$u(k) = K_p * e(k) + K_i * \sum_{i=0}^{k} e(i) * \Delta t + K_d * \frac{e(k)-e(k-1)}{\Delta t}$$

- **Rời rạc hóa bước P:** P phản ứng trực tiếp với sai số hiện tại, vì sai số ở thời điểm k đang là một giá trị số rời rạc e(k), nên khâu P không cần biến đổi phức tạp nên chỉ việc nhân với hệ số K_p:

$$P(k) = K_p * e(k)$$

- **Rời rạc hóa cho bước I:** Tích phân là diện tích dưới đường cong của đồ thị sai số e(t) từ thời điểm 0 đến t. Trong hệ thống rời rạc, diện tích này được xấp xỉ bằng tổng diện tích của các hình chữ nhật theo phương pháp Euler. Mỗi hình chữ nhật có chiều rộng là thời gian trích mẫu Δ và chiều cao là sai số e(i) tại thời điểm đó. Do đó, biểu thức tích phân $\int e(t)dt$ được thay thế bằng phép tổng của tất cả các sai số từ trước đến nay nhân với Δt:

$$I(k) = K_i * \sum_{i=0}^{k} e(i) * \Delta t$$

- **Rời rạc hóa cho đạo hàm D:** Đạo hàm thể hiện tốc độ biến thiên của đường cong sai số. Trong hệ thống rời rạc, độ dốc này được xấp xỉ bằng độ chênh lệch giữa sai số ở bước hiện tại e(k) và sai số ở bước trước đó e(k-1), chia cho khoảng thời gian Δt:

$$D(k) = K_d * \frac{e(k)-e(k-1)}{\Delta t}$$

2.2. Hệ điều hành thời gian thực

2.2.1. Tổng quan về FreeRTOS

2.2.1.1. Khái niệm hệ điều hành thời gian thực (RTOS)

- Hệ điều hành thời gian thực (Real-Time Operating System - RTOS): Là hệ điều hành chuyên dụng cho các hệ thống nhúng, nơi mà các tác vụ tính toán xử lý phải được thực hiện trong một khoảng thời gian giới hạn nghiêm ngặt. Đối với RTOS, sự chính xác của hệ thống không chỉ phụ thuộc vào kết quả logic mà còn phụ thuộc vào thời điểm kết quả đó được đưa ra.
- Có hai loại về RTOS:

- Hard Real-Time: Hệ thống nghiêm ngặt, trễ một mili-giây là gây thất bại. Ví dụ: túi khí ô tô,…
- Soft Real-Time: Có thể chấp nhận sai số trễ nhỏ. Ví dụ: hệ thống IoT điều khiển nhiệt độ, hệ thống IoT điều khiển tưới tiêu trong công nghiệp.

2.2.1.2. Lý do chọn FreeRTOS cho ESP32

- Mã nguồn mở và miễn phí: FreeRTOS là một nhân hệ điều hành cực kỳ nhỏ gọn (Microkernel), ổn định và được hỗ trợ chính thức bởi Espressif trên dòng chip ESP32.
- Cơ chế lập lịch dựa trên độ ưu tiên (Priority-based Preemptive Scheduling): Hệ điều hành liên tục giám sát và cấp quyền thực thi cho tác vụ nào có độ ưu tiên cao nhất tại mọi thời điểm. Nếu một task có độ ưu tiên cao sẵn sàng chạy, nó sẽ ngay lập tức chiếm quyền điều khiển CPU từ tác vụ thấp hơn.
- Hỗ trợ đa lõi (Multicore): ESP32 có 2 lõi CPU (Core 1 và Core 2). FreeRTOS tích hợp sẵn trong ESP-IDF/Arduino framework cho phép cấu hình phân chia các Task chạy song song trên cả 2 lõi để tối ưu tối đa hiệu năng phần cứng.

2.2.1.3. Kiến trúc nhân (Kernel Architecture) của FreeRTOS.

- Bộ điều phối/Lập lịch (Scheduler): Chịu trách nhiệm phân chia thời gian xử lý của CPU cho các tác vụ.
- Quản lý bộ nhớ (Memory Management): FreeRTOS cung cấp các cơ chế cấp phát bộ nhớ động (Heap) thông qua các file cấu hình để cấp phát vùng nhớ Stack cho từng Task mà không làm tràn bộ nhớ của vi điều khiển.
- Đồng bộ và giao tiếp: Cung cấp các công cụ để các tác vụ nói chuyện và nhường nhịn nhau.

2.2.1.4. Nguyên lý khởi chạy của hệ thống FreeRTOS

- Bước 1: Khởi tạo phần cứng: Khi ESP32 cấp nguồn, chip sẽ chạy hàm setup() trước để cấu hình các ngoại vi cơ bản như là I2C cho SHT31, PWM cho DRV8833.
- Bước 2: Khởi tạo tác vụ: Tiếp theo, các hàm tạo tác vụ được gọi để khai báo các Task, cấp phát vùng nhớ Stack và gán độ ưu tiên cho vTask_Control và vTask_MQTT. Lúc này các Task được đưa vào trạng thái Ready.
- Bước 3: Kích hoạt bộ lập lịch: Hệ điều hành gọi hàm vTaskStartScheduler(). Kể từ lúc này vòng lặp tuần tự truyền thống bị vô hiệu hóa, quyền điều khiển chip hoàn toàn thuộc về bộ lập lịch của FreeRTOS, và hệ thống chính thức vận hành theo thời gian thực.

2.2.2. Quản lý đa tác vụ

2.2.2.1. Các trạng thái của một tác vụ

- Trong FreeRTOS, một tác vụ tại một thời điểm chỉ có thể nằm ở một trong hai trạng thái lớn: Running hoặc Not Running. Trạng thái không chạy được chia chi tiết thành:
- Ready: Task đã chuẩn bị xong dữ liệu và đang đợi bộ lập lịch cấp quyền chiếm dụng CPU để chạy.
- Blocked: Task đang phải tạm dừng để đợi một sự kiện thời gian (hàm vTaskDelay()) hoặc đợi ngoại vi. Lúc này, Task tự động nhường CPU cho tác vụ khác, không gây lãng phí tài nguyên.
- Suspended (Đình chỉ): Task bị đưa vào trạng thái ngủ chủ động bằng lệnh và chỉ thức dậy khi có lệnh từ Task khác.

2.2.2.2. Bộ lập lịch phân phối theo độ ưu tiên

- Độ ưu tiên: Mỗi Task khi khởi tạo được gán một mức độ ưu tiên bằng số nguyên. Bộ lập lịch luôn chọn Task có độ ưu tiên cao nhất trong danh sách Ready để chạy.
- Cơ chế chiếm quyền: Khi vTask_Control kết thúc thời gian chờ, bộ lập lịch sẽ ngay lập tức ngắt ngang vTask_MQTT, cất ngữ cảnh vào Stack để nhường quyền xử lý CPU cho Task điều khiển tính toán PID đúng chu kỳ.

2.2.2.3. Quản lý đa lõi đối xứng trên ESP32

- Hệ điều hành tận dụng cấu trúc 2 lõi CPU (Core 0 và Core 1) của chip ESP32 thông qua hàm xTaskCreatePinnedToCore():
    - Core 0: Ghim cố định vTask_MQTT đảm nhiệm việc truyền nhận dữ liệu qua mạng.
    - Core 1: Ghim cố định vTask_Control thực hiện nhiệm vụ đọc cảm biến SHT31 và chạy thuật toán PID.

2.2.3. Cơ chế giao tiếp nội bộ

2.2.3.1. Khái niệm về giao tiếp nội bộ trong RTOS

Trong một hệ thống chạy hệ điều hành thời gian thực, các tác vụ không hoạt động biệt lập mà luôn có nhu cầu trao đổi dữ liệu hoặc đồng bộ tín hiệu với nhau.

FreeRTOS cung cấp các cơ chế giao tiếp và đồng bộ tiêu chuẩn như:

- Queue : Dùng để truyền nhận dữ liệu theo dạng tin nhắn giữa các tác vụ hoặc giữa hàm ngắt (ISR) với tác vụ theo nguyên lý FIFO.

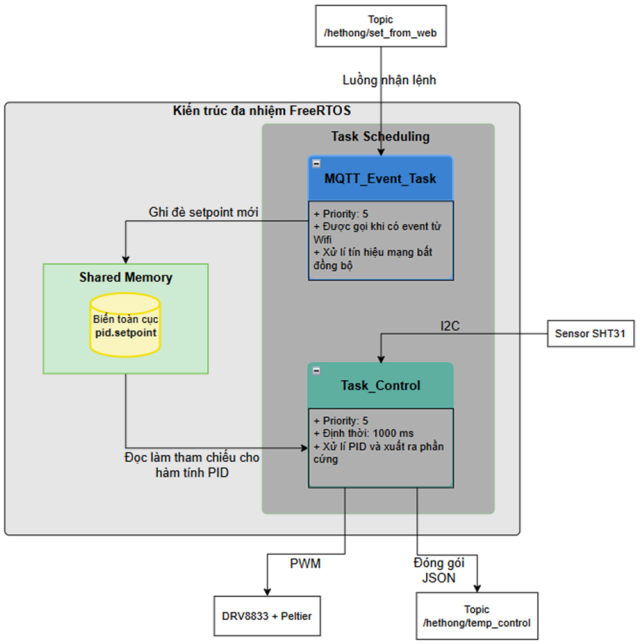
- Binary Semaphore / Mutex: Dùng để đồng bộ hóa sự kiện hoặc bảo vệ tài nguyên dùng chung (biến toàn cục, ngoại vi SPI/I2C) không bị xâm phạm cùng một lúc bởi nhiều tác vụ.

2.2.3.2. Giải pháp chia sẽ dữ liệu trong đồ án

Trong kiến trúc phần mềm hiện tại của đồ án, nhóm đã thực hiện tối ưu hóa cấu trúc firmware bằng cách phân tách độc lập hai tác vụ trên hai vùng dữ liệu tuyến tính riêng biệt:

- vTask_Control nhận giá trị nhiệt độ đặt từ biến toàn cục Setpoint để tính toán PID.
- vTask_MQTT nhận dữ liệu thay đổi từ Server và cập nhật trực tiếp vào biến Setpoint.

Do kiến trúc truyền nhận ở biên được thiết kế theo dạng Một tác vụ ghi (Task MQTT) và Một tác vụ đọc (Task Control) trên một biến cấu trúc nguyên tử đơn giản, hệ thống hoàn toàn loại bỏ được nguy cơ xung đột tài nguyên. Việc không sử dụng các khối Queue hay Semaphore giúp giảm thiểu tối đa bộ nhớ RAM cấp phát, triệt tiêu thời gian trễ do cơ chế block/unblock của hàng đợi, giúp hệ thống hoạt động với hiệu suất tối ưu và độ tin cậy cao nhất.



Hình 1.1: Sơ đồ chia sẻ dữ liệu giữa các task

2.3. Giao thức truyền thông & mạng

2.3.1. Giao thức MQTT

2.3.1.1. Khái niệm về giao thức MQTT

- MQTT (Message Queuing Telemetry Transport): Là giao thức truyền thông mã nguồn mở, hoạt động ở tầng ứng dụng trên nền tảng của bộ giao thức TCP/IP.
- Giao thức này được thiết kế theo hướng cực kỳ tối giản, gọn nhẹ , sử dụng băng thông thấp và tốn rất ít năng lượng. Do đó, nó hoàn toàn phù hợp với các vi điều khiển có cấu hình tài nguyên hạn chế như ESP32 trong đồ án.

2.3.1.2. Mô hình kiến trúc Publish/Subscribe

Thay vì sử dụng mô hình Client/Server truyền thống (nơi các thiết bị phải nói chuyện trực tiếp với nhau), MQTT sử dụng kiến trúc Publish/Subscribe thông qua một trung tâm trung chuyển trung gian:

- MQTT Broker: Là máy chủ trung tâm (Trong đồ án là phần mềm Mosquitto cài trên Raspberry Pi). Broker có nhiệm vụ nhận tất cả bản tin từ nguồn gửi, phân loại và phân phối chúng đến đúng nguồn nhận.
- MQTT Client: Là bất kỳ thiết bị nào kết nối vào Broker (Trong đồ án gồm có ESP32 và Node-RED Dashboard).
    - Publish: Hành động Client gửi một gói tin kèm theo một chủ đề cụ thể (Topic) lên Broker. (Ví dụ: ESP32 gửi dữ liệu nhiệt độ lên topic /hethong/temp).
    - Subscribe: Hành động Client đăng ký với Broker rằng mình muốn nhận dữ liệu từ một topic nào đó. Khi có dữ liệu mới trên topic đó, Broker sẽ tự động đẩy về cho Client. (Ví dụ: Node-RED đăng ký topic /hethong/temp để vẽ biểu đồ, còn ESP32 đăng ký topic /hethong/setpoint để đợi nhận lệnh thay đổi nhiệt độ từ Web).
- Topic: Là các đường dẫn định tuyến văn bản, phân cấp bằng dấu gạch chéo /, đóng vai trò như các "địa chỉ hộp thư" để Broker phân loại dữ liệu.

2.3.1.3. Các đặc tính kỹ thuật cốt lõi ứng dụng trong đồ án

- Chất lượng dịch vụ QoS: MQTT hỗ trợ 3 cấp độ QoS (0, 1, 2) để đảm bảo độ tin cậy của bản tin. Trong đồ án, nhóm cấu hình QoS 1 (At least once - Ít nhất một lần) cho bản tin cài đặt Setpoint để đảm bảo lệnh điều khiển từ Web chắc chắn phải tới được ESP32 ngay cả khi mạng bị chập chờn.

- Bản tin giữ lại: Khi ESP32 xuất bản dữ liệu với cờ *Retain* = *True*, Broker sẽ lưu lại bản tin cuối cùng này. Nhờ vậy, ngay khi người dùng vừa mở giao diện Web Node-RED lên, Dashboard sẽ ngay lập tức có dữ liệu nhiệt độ hiển thị mà không cần phải đợi đến chu kỳ gửi tiếp theo của ESP32.

2.3.1.4. Lý do lựa chọn MQTT cho đồ án

- Tối ưu băng thông và phần cứng: Cấu trúc khung tiêu đề (Header) của gói tin MQTT cực kỳ nhỏ (chỉ từ 2 Byte), nhẹ hơn rất nhiều so với giao thức HTTP truyền thống (thường ngốn hàng trăm Byte cho mỗi Request), giúp ESP32 tiết kiệm RAM và thời gian xử lý để tập trung chạy tác vụ PID.
- Khả năng đáp ứng thời gian thực (Real-time): Cơ chế kết nối duy trì liên tục (Keep-Alive) và chủ động đẩy dữ liệu từ Broker giúp các lệnh thay đổi Setpoint từ Web được thực thi ở biên gần như ngay lập tức với độ trễ mili-giây.

2.3.2. Hạ tầng mạng ảo (Mesh VPN) & NAT Traversal

2.3.2.1. Rào cản từ mạng nội bộ và cơ chế NAT

- **Thách thức:** Thiết bị biên ESP32 và máy chủ Raspberry Pi đặt tại một địa điểm nào đó và sử dụng địa chỉ IP nội bộ Private IP. Cơ chế NAT trên Router đóng vai trò như tường lửa, chặn toàn bộ kết nối từ Internet đi vào mạng cục bộ.
- Vì vậy, người dùng không thể truy cập trực tiếp vào máy chủ Node-RED hay MQTT Broker từ mạng bên ngoài.

2.3.2.2. Kỹ thuật đục lỗ NAT (NAT Traversal)

- Để giải quyết rào cản trên mà không cần cấu hình mở cổng Router (Port Forwarding) kém an toàn, đồ án áp dụng kỹ thuật NAT Traversal (Đục lỗ NAT).
- Cơ chế này sử dụng các máy chủ trung gian để giúp Client (Laptop) và Gateway (Raspberry Pi) trao đổi "tọa độ mạng" công cộng của nhau, từ đó tự động mở một đường truyền trực tiếp xuyên qua các tầng bảo mật của Router.

2.3.2.3. Giải pháp mạng ảo dạng lưới (Mesh VPN) qua Tailscale

- Đề tài ứng dụng giải pháp Tailscale dựa trên nền tảng giao thức mã hóa WireGuard để tạo ra một mạng ảo dạng lưới (Mesh VPN) phi tập trung.
- Cơ chế hoạt động: Sau khi đục lỗ NAT thành công, dữ liệu điều khiển sẽ đi theo đường thẳng trực tiếp (Peer-to-Peer) giữa Laptop và Raspberry Pi mà

không cần bắc cầu qua máy chủ trung tâm, giúp giảm thiểu độ trễ truyền tin xuống mức thấp nhất.

- Hiệu quả: Mỗi thiết bị trong mạng lưới được cấp một địa chỉ IP ảo cố định (dạng 100.x.x.x), cho phép Vũ giám sát và cấu hình Setpoint nhiệt độ an toàn từ bất kỳ đâu qua Internet với độ bảo mật tuyệt đối.

2.4. Tổng quan về các chuẩn giao tiếp phần cứng

2.4.1. Giao tiếp I2C

2.4.1.1. Khái niệm và Kiến trúc vật lý của I2C

- Khái niệm: I2C là chuẩn giao tiếp đồng bộ, nối tiếp, dạng bán song công (Half-duplex), được sử dụng để truyền dữ liệu khoảng cách ngắn giữa vi xử lý với các IC ngoại vi.
- Kiến trúc phần cứng: Giao tiếp I2C hoạt động theo mô hình Master-Slave. Chuẩn này cực kỳ tiết kiệm chân phần cứng khi chỉ sử dụng duy nhất 2 đường dây tín hiệu:
    - SCL (Serial Clock Line): Đường xung nhịp đồng hồ do chip Master (ESP32) phát ra để đồng bộ hóa việc truyền dữ liệu.
    - SDA (Serial Data Line): Đường truyền dữ liệu hai chiều giữa Master và Slave.
- Cấu trúc mạch: Cả hai đường dây SCL và SDA đều được thiết kế theo dạng cực thu hở (Open-drain) nên bắt buộc phải có điện trở kéo lên nguồn (Pull-up Resistors) để duy trì trạng thái logic HIGH mặc định trên đường truyền.

2.4.1.2. Nguyên lý truyền nhận dữ liệu cơ bản

Quá trình truyền một khung dữ liệu trên đường bus I2C diễn ra theo các bước nghiêm ngặt:

- Điều kiện START: Master kéo đường SDA xuống mức LOW trong khi SCL vẫn đang ở mức HIGH để báo hiệu bắt đầu phiên truyền thông.
- Địa chỉ Slave: Master gửi chuỗi 7-bit địa chỉ của thiết bị Slave mà nó muốn nói chuyện kèm theo 1-bit R/W.
- Phản hồi ACK/NACK: Thiết bị Slave có địa chỉ trùng khớp sẽ kéo đường SDA xuống mức LOW để phản hồi lại cho Master biết nó đã sẵn sàng.
- Truyền dữ liệu: Dữ liệu được truyền nối tiếp từng bit một (8-bit cho mỗi byte). Sau mỗi byte, bên nhận lại phải phát 1-bit ACK để xác nhận.
- Điều kiện STOP: Master kéo đường SDA từ mức LOW lên HIGH trong khi SCL đang ở mức HIGH để kết thúc phiên làm việc và giải phóng đường bus.

2.4.1.3. Ứng dụng và tính năng nâng cao trong đồ án (SHT31)

- Truy xuất dữ liệu cảm biến: ESP32 đóng vai trò Master chủ động gửi lệnh kích hoạt phép đo qua I2C, sau đó đọc về 6 byte dữ liệu từ SHT31 (gồm 2 byte nhiệt độ thô, 1 byte CRC kiểm lỗi nhiệt độ, 2 byte độ ẩm thô và 1 byte CRC kiểm lỗi độ ẩm).
- Cơ chế kiểm lỗi CRC (Cyclic Redundancy Check): Đây là điểm cộng kỹ thuật lớn. Mã CRC 8-bit đi kèm giúp ESP32 kiểm tra tính toàn vẹn của dữ liệu nhận về. Nếu phát hiện dữ liệu bị nhiễu trên đường dây I2C làm sai lệch thuật toán mã hóa, ESP32 sẽ tự động hủy gói tin đó và đợi chu kỳ kế tiếp, đảm bảo thuật toán PID không bị tính toán sai dựa trên số liệu "rác".

2.4.2. Kỹ thuật băm xung (PWM)

2.4.2.1. Tổng quan và khái niệm PWM

- Vi điều khiển ESP32 chỉ hoạt động với hai mức logic số: mức High (3.3V) và mức Low(0V). Tuy nhiên, các thiết bị chấp hành như động cơ, sò nhiệt hay đèn LED cần một điện áp biến đổi liên tục từ 0V đến mức cực đại để thay đổi tốc độ, công suất nhiệt hay độ sáng. Để giải quyết bài toán này mà không cần dùng đến các mạch biến đổi tín hiệu tương tự phức tạp (DAC), kỹ thuật điều chế độ rộng xung (PWM - Pulse Width Modulation) được nhóm sử dụng.
- Bản chất của PWM là việc bật và tắt một tín hiệu điện áp ở tốc độ cực kỳ nhanh. Có ba thông số quan trọng để tạo nên một tín hiệu PWM đó là:
    - **Chu kỳ xung (T_{PWM}):** Là khoảng thời gian cần thiết để một xung thực hiện trọn vẹn một chu trình.
    - **Tần số băm xung (f_{PWM}):** Thể hiện số lần tín hiệu bật/tắt trong vòng 1 giây.
    - **Chu kỳ nhiệm vụ (Duty Cycle - D):** Là tỉ lệ phần trăm giữa thời gian tín hiệu ở mức High (t_{ON}) so với tổng thời gian của một chu kỳ (T_{PWM}). Công thức tính như sau:

$$D(\%) = \frac{t_{ON}}{T_{PWM}} * 100\%$$

2.4.2.2. Ứng dụng cho hệ thống của nhóm

- Điện áp thực tế mà sò Peltier nhận được ($V_{sò}$) sẽ phụ thuộc vào Duty Cycle với công thức tính như sau:

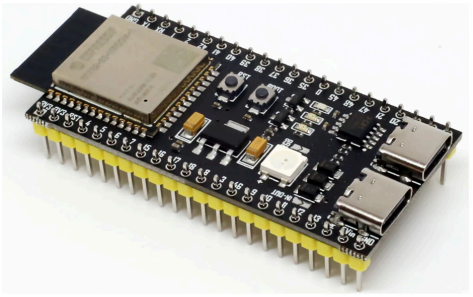
$$V_{sò} = V_{max} * \frac{D}{100}$$

VD: nếu cấp điện áp 5v cho sò và PWM Duty Cycle nhận được là 50% thì sò sẽ hoạt động ở mức 2.5v.

- Tiếp đến nhóm sẽ cấu hình tần số băm xung ở mức 18 KHz, giúp sò không bị phát ra tiếng rít tần số cao, hoạt động êm khi sử dụng các tần số thấp.
- Sử dụng độ phân giải 12 bit cho các giá trị PWM, chia nhỏ dải Duty Cycle 0-100% thành 4096 mức điện áp khác nhau từ 0-4095.

2.5. Linh kiện cốt lõi sử dụng

2.5.1. Vi điều khiển ESP32-S3 - MCU



ESP32-S3-DevKit-1

- Sở hữu kiến trúc lõi kép mạnh mẽ, ESP32-S3 cho phép chạy hệ điều hành thời gian thực FreeRTOS mượt mà.
- Tích hợp sẵn module Wi-Fi giúp hệ thống dễ dàng kết nối trực tiếp với MQTT Broker trên Raspberry Pi.
- Khối LEDC (LED Control) tích hợp trong ESP32 hỗ trợ băm xung PWM phần cứng với độ phân giải lên đến 12-bit ở tần số cao 18kHz.

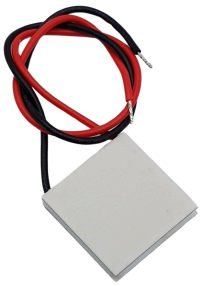
2.5.2. Cảm biến SHT31 - Sensor



Cảm biến SHT31

- SHT31 (Cảm biến nhiệt độ, độ ẩm): đo nhiệt độ và độ ẩm hiện tại của môi trường và gửi dữ liệu này về cho ESP32 qua giao thức I2C.
- Thông số kỹ thuật:
    - Cảm biến: Sensiron SHT31-D.
    - Đo lường: Nhiệt độ và Độ ẩm.
    - Độ chính xác (Độ ẩm): ±2% (độ ẩm tương đối). + Độ chính xác (Nhiệt độ): ±0.3°C.
    - Giao tiếp: I2C (true I2C) .
    - Điện áp hoạt động (Vin): 2.5V - 5V DC (tương thích logic 3V và 5V)

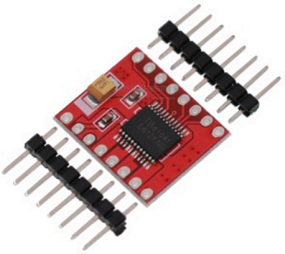
2.5.3. Sò Peltier (TEC) - Actuator



Sò nóng lạnh 5v

- Hoạt động dựa trên hiệu ứng nhiệt điện. Khi cấp dòng điện một chiều đi qua, một mặt của sò sẽ hấp thụ nhiệt để làm lạnh và mặt kia sẽ tỏa nhiệt. Đặc biệt, nếu đảo chiều dòng điện, chức năng của hai mặt sẽ hoán đổi cho nhau.
- Sò Peltier là thành phần actuator cho hệ thống, thay đổi môi trường nhiệt độ thực tế bên trong hộp sử dụng thuật toán PID.

2.5.4. Mạch cầu H DRV8833



Mạch cầu H DRV8833

- Chịu được dòng điện lớn và có tần số đóng ngắt Mosfet cao, cực kỳ phù hợp cho PWM.
- Nhận tín hiệu xung PWM từ ESP32, điều tiết và có thể đảo cực điện áp từ nguồn cấp cho sò Peltier chuyển đổi hai mặt nóng/lạnh cho sò.

2.5.5. Raspberry Pi 4



Raspberry Pi 4

- Nhóm sử dụng máy tính nhúng Raspberry Pi làm Server trung tâm điều phối toàn bộ hệ thống (IoT Gateway).
- Sở hữu chip xử lý ARM Cortex đa nhân, RAM dung lượng lớn, có thể chạy nhiều dịch vụ mạng cùng lúc mà không có độ trễ.

Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

- **Đo lường và thu thập dữ liệu:**
    - Vi điều khiển ESP32-S3 thu thập dữ liệu nhiệt độ từ môi trường thực tế thông qua cảm biến SHT31 giao tiếp bằng chuẩn I2C.
    - Mặc dù cảm biến có hỗ trợ đo độ ẩm, nhưng để tối ưu hóa tốc độ xử lý trên bus I2C và tập trung toàn bộ tài nguyên cho bài toán nhiệt độ, nhóm loại bỏ việc đọc và xử lý dữ liệu độ ẩm.
- **Xử lý thuật toán và điều khiển công suất:**
    - Hệ thống phải tính toán được sai lệch nhiệt độ hiện tại (Error) so với giá trị Setpoint.
    - Dựa trên Error, hệ thống sử dụng thuật toán PID kết hợp logic If-Else để xuất xung PWM điều khiển mạch cầu H DRV8833. Qua đó thay đổi công suất và chiều dòng điện qua sò nhiệt Peltier.
    - Có khả năng tự động chuyển đổi linh hoạt giữa hai chế độ LÀNH (COOL) và NÓNG (HEAT) tùy thuộc vào môi trường.
- **Bảo vệ phần cứng:**
    - Do sò Peltier nhạy cảm với ứng suất nhiệt và sẽ bị phá vỡ cấu trúc nếu đảo chiều dòng điện đột ngột.
    - Vì thế phải tích hợp biện pháp an toàn: tắt hoàn toàn PWM trong 5s trước khi kích hoạt chiều dòng điện ngược lại.
- **Truyền thông và giám sát từ xa:**
    - Truyền tải dữ liệu phi tập trung qua sóng Wi-Fi dựa trên giao thức MQTT.
    - Hệ thống yêu cầu một máy chủ cục bộ với Raspberry Pi đóng vai trò Gateway chạy Mosquitto Broker và Node-RED Server để quản lý luồng dữ liệu, vẽ biểu đồ và xử lý lệnh cấu hình từ xa trên giao diện web.
    - Cho phép người dùng thiết lập giá trị Setpoint mới từ xa thông qua giao diện dashboard web.

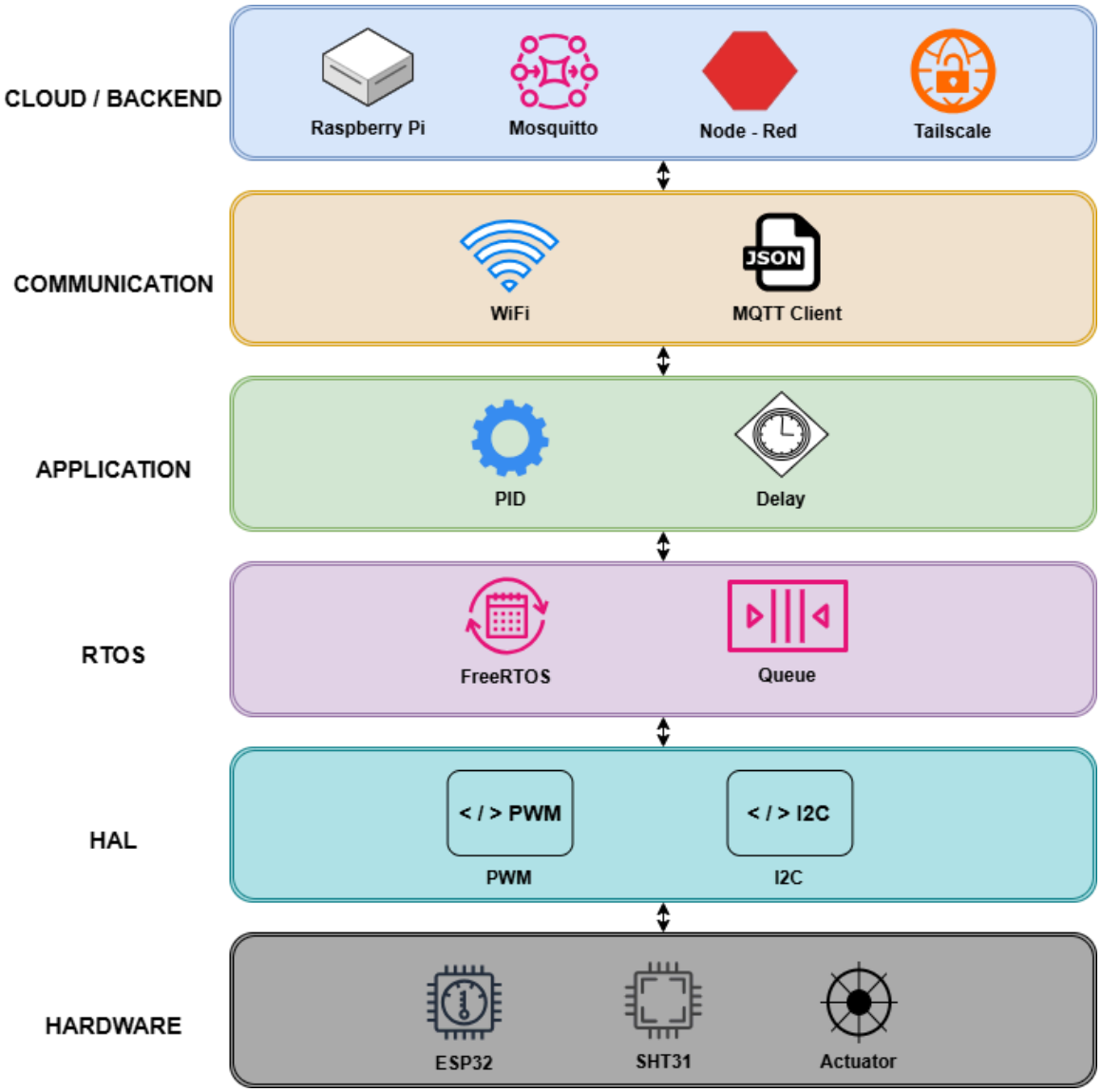
3.1.2. Yêu cầu phi chức năng

- **Tính real-time:**

    - Chu trình điều khiển nhiệt độ phải được diễn ra định kỳ và chính xác. Hệ thống sử dụng hệ điều hành FreeRTOS điều phối tác vụ chính xác cao.
    - Luồng xử lý mạng nhận lệnh từ web không được phép làm treo hoặc làm trễ luồng xử lý phần cứng.
- **Độ ổn định và triệt tiêu sai số:**
    - Thuật toán điều khiển PID phải xử lý hiện tượng bão hòa tích phân (Anti-windup) và hiện tượng vượt lố. Nhiệt độ thực tế phải bám sát Setpoint với sai số dao động nhỏ nhất có thể trong khoảng $\pm$ 5 độ C.
- **Tối ưu hóa tài nguyên hệ thống:**
    - Hệ thống phải tiết kiệm bộ nhớ RAM và Flash. vi điều khiển ESP32-S3 dồn toàn lực vào xử lý toán học (PID) và giao thức TCP/IP.
- **Khả năng bảo mật và triển khai linh hoạt:**
    - Do rào cản từ tường lửa của mạng Wi-Fi công cộng, nên hệ thống yêu cầu một giải pháp mạng ảo bảo mật cao.
    - Sử dụng giải pháp mạng Mesh VPN WireGuard thông qua nền tảng Tailscale để cung cấp IP tĩnh, giúp đường truyền được mã hóa đầu cuối. Người dùng có thể truy cập hệ thống từ bất kỳ đâu mà không cần thiết lập NAT Port Forwarding trên Router.

3.2. Sơ đồ khối tổng thể của hệ thống

3.2.1. Sơ đồ kiến trúc phân tầng hệ thống



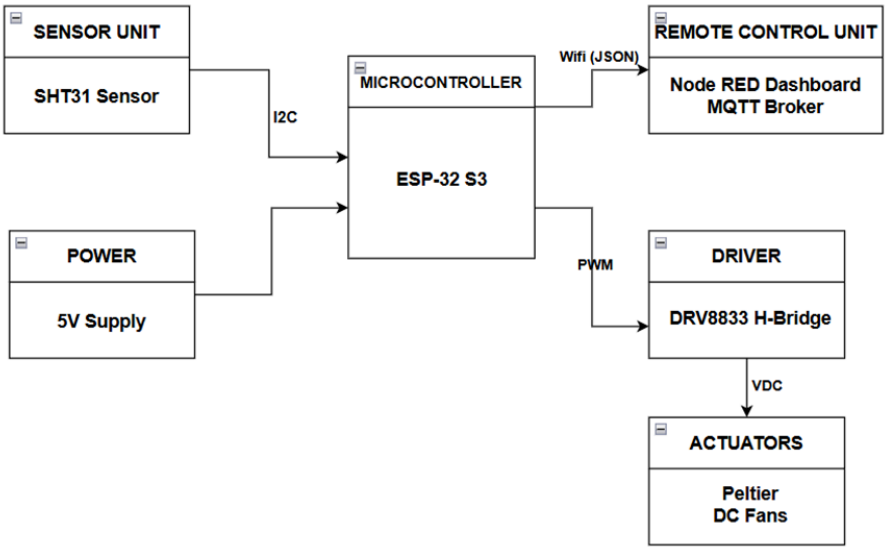
Hình 1.2: Sơ đồ kiến trúc phân tầng hệ thống

Kiến trúc hệ thống được thiết kế theo mô hình phân lớp (Layered Architecture), bao gồm 6 tầng riêng biệt, giúp tăng tính module hóa và dễ dàng bảo trì, nâng cấp:

- Tầng Hardware: Lớp dưới cùng bao gồm vi điều khiển trung tâm ESP32, cảm biến SHT31 để thu thập dữ liệu và cơ cấu chấp hành gồm mạch cầu HDRV8833 và sò nhiệt điện Peltier.

- Tầng HAL: Lớp trừu tượng hóa phần cứng, chịu trách nhiệm giao tiếp trực tiếp với linh kiện vật lý thông qua chuẩn I2C (đọc SHT31) và xuất xung PWM (điều khiển DRV8833).
- Tầng RTOS: Sử dụng nhân FreeRTOS làm lõi điều phối. Tầng này cung cấp các cơ chế quản lý đa tác vụ (Task Management), phân chia ưu tiên và định thời (vTaskDelay) chính xác để đảm bảo tính thời gian thực.
- Tầng Application: Chứa thuật toán điều khiển trọng tâm của đồ án là bộ điều khiển PID. Tầng này nhận dữ liệu từ cảm biến, tính toán sai số và ra quyết định công suất bù nhiệt.
- Tầng Communication: Đảm nhiệm việc kết nối module Wi-Fi và đóng gói/giải mã các bản tin theo chuẩn MQTT Client dưới định dạng chuỗi JSON.
- Tầng Cloud / Backend: Lớp trên cùng được triển khai trên máy tính nhúng Raspberry Pi, đóng vai trò là Gateway trung tâm. Nó chạy MQTT Broker để trung chuyển dữ liệu, Node-RED để thiết kế Web Dashboard và tích hợp Tailscale để cho phép điều khiển, giám sát an toàn từ xa qua Internet.

3.2.2. Sơ đồ kết nối phần cứng



Hình 1.3: Sơ đồ kết nối phần cứng tổng thể

3.2.3. Chức năng chi tiết các khối trong hệ thống

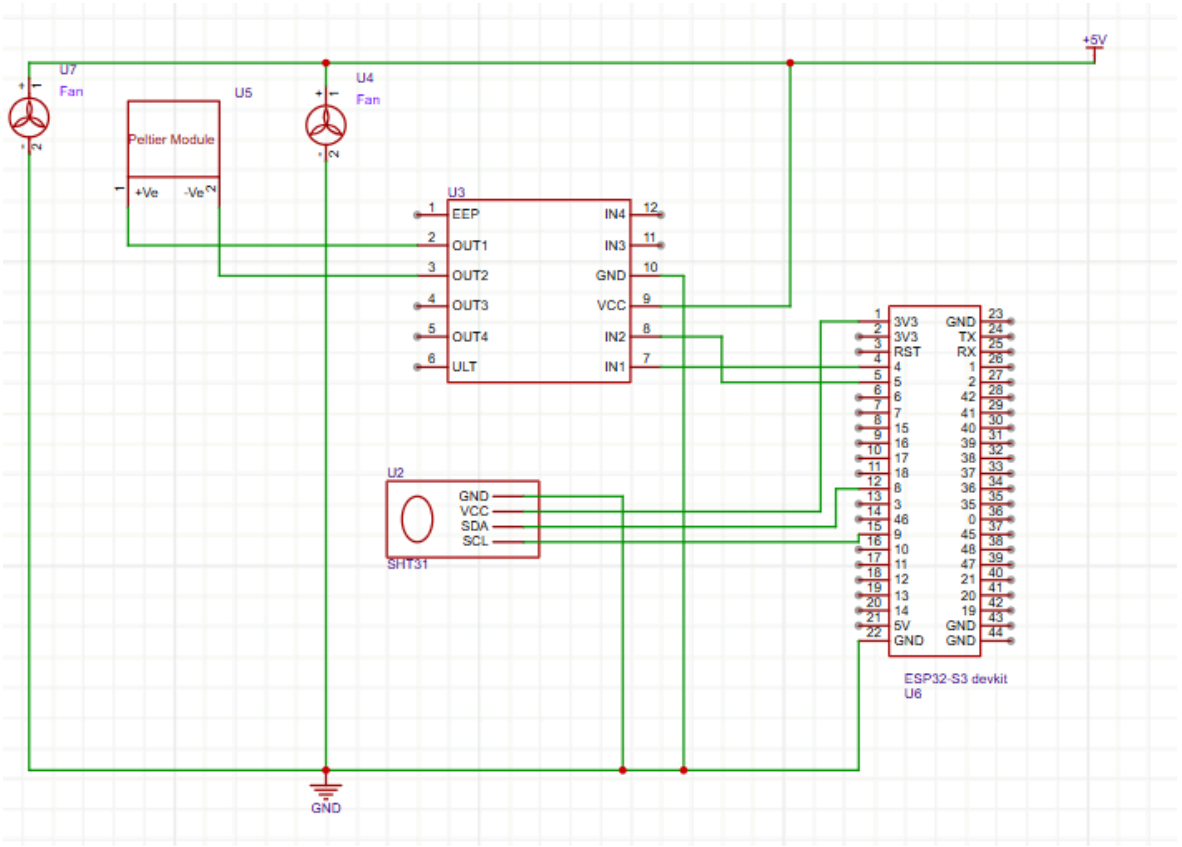
- **Khối xử lý trung tâm (ESP32):** Là bộ não điều khiển tại biên. Chịu trách nhiệm khởi tạo các ngoại vi, cấu hình nhịp thời gian thực FreeRTOS, thực hiện chu kỳ lấy mẫu từ cảm biến, tính toán thuật toán PID và xuất chuỗi xung điều khiển công suất.
- **Khối cảm biến (SHT31):** Liên tục đo đạc các thông số vật lý của môi trường nhiệt độ và độ ẩm, số hóa dữ liệu và gửi về khối xử lý trung tâm qua giao tiếp số.
- **Khối mạch lái công suất (DRV8833):** Tiếp nhận tín hiệu điều khiển dòng nhỏ xung PWM từ ESP32, khuếch đại dòng điện và điện áp từ nguồn động lực để cấp cho tải, đồng thời thực hiện đảo chiều dòng điện theo lệnh để chuyển đổi chế độ Nóng/Lạnh.
- **Khối cơ cấu chấp hành (Sò Peltier):** Chuyển hóa điện năng thành nhiệt năng dựa trên hiệu ứng nhiệt điện, trực tiếp làm thay đổi trạng thái nhiệt độ của buồng cách nhiệt.
- **Khối nguồn:** Nguồn động lực 5V này được chia làm hai nhánh. Nhánh thứ nhất cấp thẳng vào mạch cầu H DRV8833 để cung cấp dòng điện cho cơ cấu chấp hành (sò Peltier), đảm bảo nằm trong ngưỡng chịu tải an toàn của IC. Nhánh thứ hai được đưa vào khối xử lý ESP32, đi qua IC ổn áp LDO tích hợp sẵn trên board để hạ áp xuống mức 3.3V, tạo ra một đường điện áp cực kỳ ổn định và chống nhiễu để nuôi vi điều khiển trung tâm và cảm biến SHT31.

- **Khối Gateway và Máy chủ (Raspberry Pi):** Đóng vai trò là trung tâm quản lý dữ liệu tầm cao. Chạy MQTT Broker để điều phối các gói tin, vận hành Node-RED để cung cấp giao diện Web Dashboard cho người dùng và thiết lập lớp mạng ảo Tailscale để xử lý bảo mật từ xa.

3.2.4. Bản chất luồng tín hiệu và truyền thông giữa các khối

- Giao tiếp Biên - Ngoại vi: Luồng tín hiệu giữa khối cảm biến và ESP32 tuân theo chuẩn giao tiếp số đồng bộ I2C (đường truyền SCL, SDA). Luồng tín hiệu giữa ESP32 và khối mạch lái công suất là tín hiệu băm xung PWM điều khiển góc mở bán dẫn.
- Giao tiếp Không dây biên: Luồng dữ liệu từ ESP32 gửi lên Gateway sử dụng kết nối Wi-Fi truyền tải các gói tin định dạng chuỗi JSON thông qua giao thức MQTT gọn nhẹ.
- Luồng mạng diện rộng: Dữ liệu giám sát và lệnh cài đặt Setpoint từ thiết bị quản trị (Laptop/Điện thoại) đi xuyên qua không gian Internet, được mã hóa an toàn bằng giao thức WireGuard của Tailscale VPN để đi thẳng tới máy chủ Raspberry Pi mà không bị chặn bởi tường lửa NAT.

3.3. Thiết kế chi tiết phần cứng



Hình 1.4: Schematic thiết kế phần cứng

3.3.1. Cảm biến SHT31

- Kết nối 2 chân dữ liệu SCL/ SDA từ SHT31 vào chân ngoại vi của ESP32-S3.
- Sử dụng điện trở kéo lên của ESP32-S3 cho 2 chân SCL/SDA.
- SHT32 hoạt động ở mức điện áp 3.3V nên sẽ cắm chân VCC của cảm biến vào chân 3.3V của ESP32, chân GND thì nối chung mass toàn mạch.

3.3.2. Mạch cầu H DRV8822 và sò Peltier

- Tín hiệu PWM điều khiển NÓNG/LẠNH xuất từ chân GPIO của ESP32 và đưa vào chân IN1/ IN2 của DRV8833.
- Chân VM (power) của DRV8833 nối với nguồn tổ ong 5v.
- Hai chân ngõ ra OUT1 và OUT2 của DRV8833 nối trực tiếp vào 2 dây nguồn của sò Peltier.

3.3.3. Nguyên tắt chung mass và nguồn cấp

- Nhóm thiết kế một đường điện áp GND kết nối xuyên suốt tất cả các module: chân GND của nguồn 5V, chân GND của ESP32-S3, DRV8833 và SHT31.
- Dòng điện nuôi Peltier sẽ chạy từ nguồn 5V qua DRV8833 và đi thẳng về mass của nguồn 5V mà không chạy xuyên qua board mạch ESP32-S3, bảo vệ an toàn cho vi điều khiển ESP32.

3.4. Thiết kế chi tiết phần mềm

3.4.1. Sơ đồ kiến trúc phần mềm và phân bổ tác vụ

- Hàm app_main() (Khởi tạo hệ thống): Khởi tạo tuần tự các lớp ngoại vi: I2C (Cảm biến) → PWM (IC Công suất) → Wi-Fi → Khởi động giao thức MQTT. Sau khi nền tảng sẵn sàng, hệ thống cấp phát vùng nhớ và kích hoạt Tác vụ RTOS.
- Tác vụ Task_Control là luồng thực thi chính:
    - Chạy tuần hoàn với chu kỳ thời gian thực chính xác 1 giây.

- Đảm nhiệm toàn bộ vòng lặp khép kín: Đọc cảm biến → Chạy thuật toán PID → Băm xung Peltier → Đóng gói và xuất bản dữ liệu JSON lên MQTT Broker.

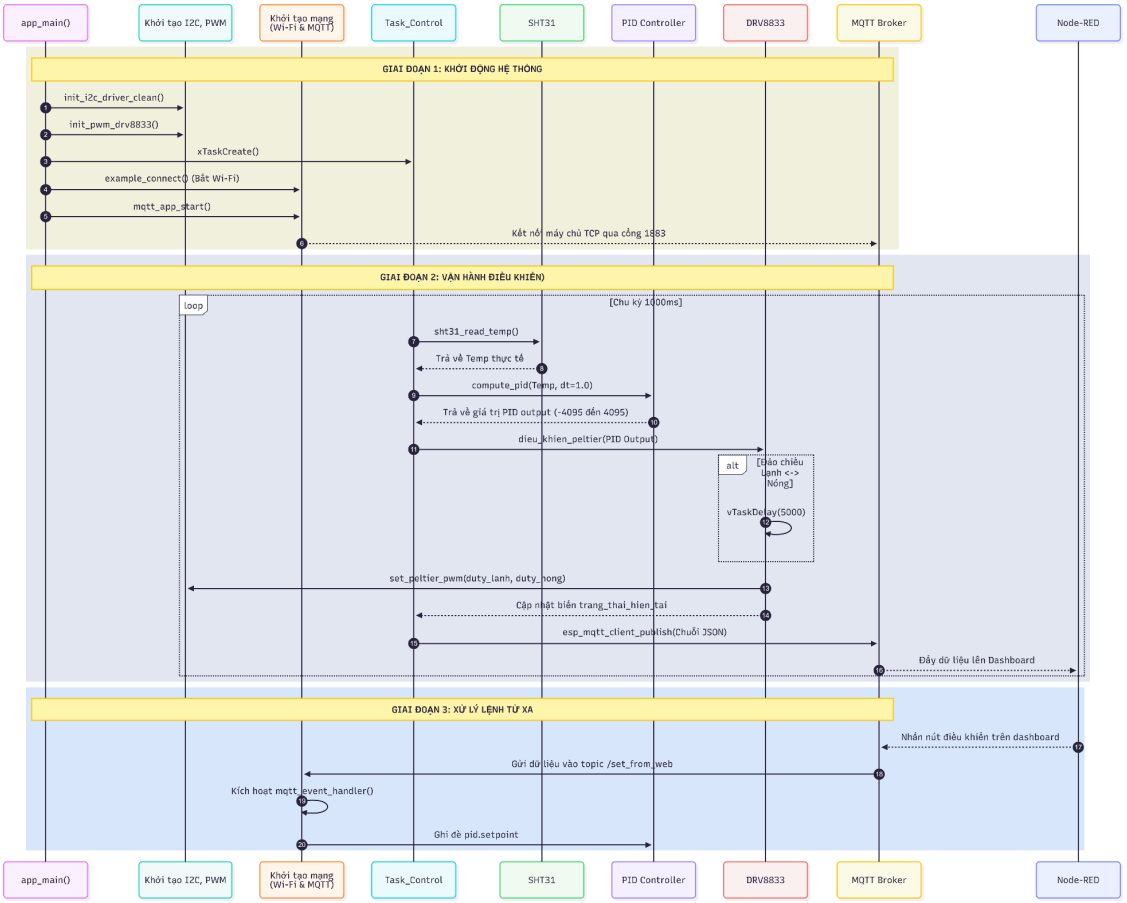
- Trình xử lý sự kiện mqtt_event_handler:
  - Hoạt động ngầm như một hàm callback bất đồng bộ. Mỗi khi người dùng thay đổi Setpoint trên Web, hàm này sẽ đánh thức, giải mã chuỗi dữ liệu nhận được và cập nhật trực tiếp vào biến bộ nhớ dùng chung, cung cấp mục tiêu mới cho tác vụ PID.

3.4.2. Lưu đồ thuật toán điều khiển chính

Vòng lặp của Task_Control vận hành theo lưu đồ 6 bước:

- Thu thập dữ liệu: ESP32 truy xuất thanh ghi I2C để lấy giá trị nhiệt độ (temp) và độ ẩm (hum) thực tế từ SHT31.
- Xử lý ngoại lệ: Đánh giá tính toàn vẹn của kết nối I2C. Nếu mất tín hiệu cảm biến, hệ thống ép nhiệt độ về mức cảnh báo -45.0°C, ngắt lập tức xung PWM để tránh rủi ro quá nhiệt cho hệ thống phần cứng và reset bộ tích phân PID.
- Tính toán điều khiển: Đưa sai số giữa nhiệt độ thực tế và biến Setpoint (do MQTT quản lý) vào hàm compute_pid() để trích xuất ra giá trị công suất bù nhiệt.
- Xét dấu và Chuyển mạch:
  - PID âm → Kích hoạt kênh Làm lạnh, tắt Làm nóng.
  - PID dương → Kích hoạt kênh Làm nóng, tắt Làm lạnh.
- Truyền thông IoT: Đóng gói toàn thông số (temp, hum, set, pwm, mode) thành chuỗi JSON và gửi lên Topic /hethong/temp_control.
- Định thời: Gọi hàm vTaskDelay nghỉ đúng 1000ms để đảm bảo tần số lấy mẫu cố định cho thuật toán điều khiển.

3.4.3. Luồng hoạt động chính của hệ thống

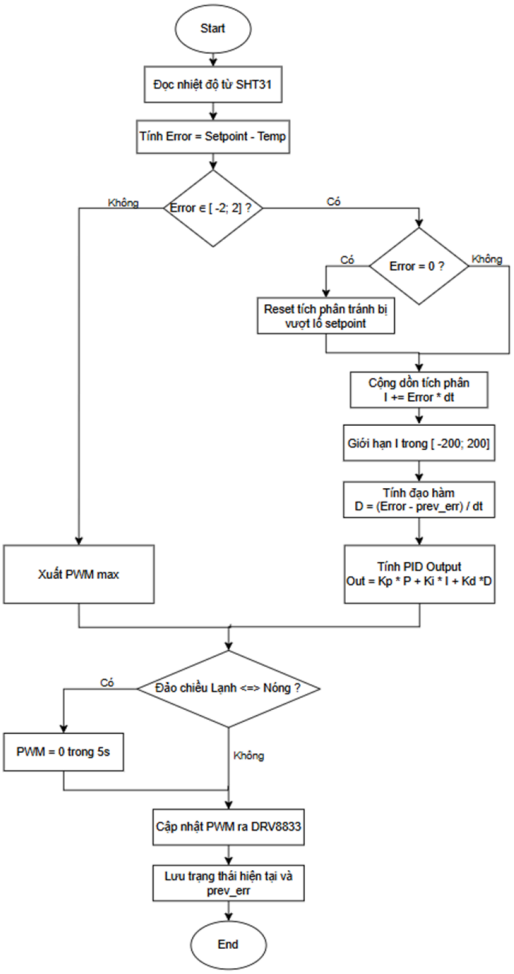


Hình 1.5: Sequence Diagram của hệ thống

- **Giai đoạn 1:** Hàm **app_main()** đóng vai trò chuẩn bị, tuần tự gọi các hàm khởi tạo giao tiếp ngoại vi **(init_i2c, init_pwm)**, khởi tạo ngăn xếp mạng Wi-Fi và giao thức MQTT. Sau khi các tài nguyên đã sẵn sàng, hệ điều hành FreeRTOS cấp phát bộ nhớ và gọi **xTaskCreate()** để sinh ra tác vụ **Task_Control** và đẩy hệ thống vào trạng thái hoạt động đa nhiệm.
- **Giai đoạn 2:** Giai đoạn này sẽ vận hành điều khiển thông qua vòng lặp kín diễn ra liên tục với mỗi 1s trong lõi **Task_Control**. Thực hiện từng bước sau:
    - Vi điều khiển ESP32-S3 gửi lệnh đọc đến cảm biến SHT31 và chờ trả về nhiệt độ thực tế.
    - Dữ liệu này được đưa vào bộ điều khiển PID để tính toán sai số và xuất ra giá trị output từ -4095 đến 4095.

    - Hàm **dieu_khien_peltier()** phân giải giá trị này thành lệnh băm xung và chiều dòng điện, cập nhật xuống mạch cầu H DRV8833 để tác động lên sò nhiệt.
    - Cuối cùng, ESP32 sẽ đóng gói toàn bộ trạng thái thành chuỗi JSON và đẩy dữ liệu lên MQTT Broker để đẩy về Node-RED. Tác vụ sau đó gọi **vTaskDelay()** nghỉ trong khoảng 1000ms.
- **Giai đoạn 3:** Giai đoạn này xử lí lệnh từ xa, hoạt động bất đồng bộ. Khi người dùng thao tác đổi setpoint trên giao diện của Node-RED, lệnh sẽ được đẩy xuống MQTT Broker và truyền về ESP32. Bộ xử lý sự kiện **mqtt_event_handler** được kích hoạt, ghi đè giá trị **pid.setpoint** mới vào bộ nhớ dùng chung. Ở chu kỳ 1000ms tiếp theo của **Giai đoạn 2**, bộ PID sẽ tự động lấy setpoint mới này để tính toán mà không làm treo hay gián đoạn hệ thống.

3.4.4. Luồng hoạt động và các cơ chế tối ưu thuật toán PID trong hệ thống



Hình 1.6: Sơ đồ thuật toán PID

Nhằm đáp ứng tiêu chuẩn điều khiển tự động, phương trình PID trong mã nguồn được tích hợp 3 kỹ thuật chống nhiễu:

- Chống tích lũy sai số: Khâu tích phân (I) bị kẹp cứng biên độ tại ngưỡng [-200.0, 200.0]. Kỹ thuật này triệt tiêu hiện tượng quá chấn (Overshoot) trầm trọng khi hệ thống phải đáp ứng những bước nhảy Setpoint lớn.
- Vùng chết tích phân: Thuật toán chỉ cho phép khâu I bắt đầu cộng dồn sai số khi nhiệt độ thực tế đã tiến sát mục tiêu (sai số nhỏ hơn 2.0°C). Điều này giúp hệ thống cập bến Setpoint êm ái.
- Xóa tích phân khi đảo chiều: Khi đường nhiệt độ cắt ngang qua đường Setpoint (sai số đổi dấu), giá trị tích lũy lập tức bị xóa về 0 để giải phóng quán tính nhiệt dư thừa.

3.4.5. Cơ chế bảo vệ phần cứng và bộ lọc dữ liệu đám mây

- Bộ lọc dữ liệu từ xa: Vì hệ thống hoàn toàn nhận lệnh từ Internet, mã nguồn tích hợp một chốt chặn phần mềm trong hàm MQTT Event. Giá trị Setpoint từ Web tải về chỉ được ghi nhận nếu nằm trong dải an toàn 10.0°C – 40.0°C và có sự sai khác lớn hơn 0.1 so với trạng thái cũ. Điều này giúp loại bỏ các lệnh nhiễu (Spam) từ mạng truyền thông.
- Bảo vệ sốc nhiệt: Trong khâu xuất xung PWM, mỗi khi hệ thống lật trạng thái Nóng/Lạnh, vi điều khiển sẽ ngắt điện áp tải và ép hệ thống chờ 5 giây. Cơ chế này mô phỏng chuẩn an toàn của lốc máy lạnh, ngăn chặn sự giãn nở đột ngột gây vỡ các phiến bán dẫn bên trong khối Peltier.

3.5. Thiết kế truyền thông và giao thức

3.5.1. Kiến trúc mạng và mô hình truyền thông MQTT

- Kết nối vật lý: Thiết bị biên (ESP32) kết nối vào mạng Wi-Fi cục bộ và giao tiếp với máy chủ thông qua địa chỉ IP tĩnh IPv4 được cấu hình cứng là 172.20.10.2 tại cổng mặc định 1883.
- Mô hình Publish/Subscribe: Sử dụng giao thức MQTT gọn nhẹ. Hệ thống giao phó việc truyền nhận cho tác vụ mqtt_event_handler và được khởi tạo thông qua hàm mqtt_app_start().
- Chất lượng dịch vụ QoS: Trong tác vụ Task_Control, lệnh gửi dữ liệu lên Broker được nhóm thiết lập cứng tham số QoS1. Điều này đảm bảo dữ liệu giám sát quan trọng chắc chắn phải được chuyển tới Broker ít nhất một lần, giúp Dashboard trên Web không bị mất dữ liệu khi đường truyền Wi-Fi chập chờn.

3.5.2. Quy hoạch cấu trúc Topic mạng

Tên TOPIC	Hướng truyền	Chức năng chi tiết trong hệ thống
/hethong/temp_control	ESP32 sang Node-RED	Publish dữ liệu giám sát định kỳ bao gồm Nhiệt độ, Độ ẩm, Setpoint hiện tại, mức PWM và Chế độ hoạt động. Chu kỳ truyền là 1 giây/lần.
/hethong/set_from_web	Node-RED sang ESP32	Subscribe để lắng nghe sự kiện thay đổi nhiệt độ mong muốn (Setpoint) từ người dùng thao tác trên giao diện Web Dashboard.

3.5.3. Định dạng cấu trúc bản tin dữ liệu (Payload Format)

3.5.3.1. Bản tin giám sát

- Sử dụng định dạng JSON để đóng gói nhiều thông số vào cùng một lần gửi, tối ưu hóa băng thông mạng.
- Cấu trúc mẫu:
  { "temp": 26.54, "hum": 62.30, "set": 25.0, "pwm": 185, "mode": "COOL"}

3.5.3.2. Bản tin điều khiển Setpoint

Thay vì dùng JSON phức tạp tốn tài nguyên giải mã, nhóm thiết kế bản tin nhận về dưới dạng chuỗi ký tự thô (Raw String) chứa duy nhất giá trị số thực.

- Cấu trúc mẫu: 23.5
- Cơ chế xử lý trong code: Khi sự kiện MQTT_EVENT_DATA được kích hoạt trên topic /hethong/set_from_web, ESP32 sẽ lưu chuỗi này vào một bộ đệm (Buffer 16 bytes), sau đó sử dụng hàm atof(buf) để chuyển đổi trực tiếp thành số thực.
- Đặc biệt: Hệ thống có tích hợp thuật toán "Chốt chặn phần mềm", chỉ chấp nhận cập nhật Setpoint mới nếu giá trị đó nằm trong dải an toàn từ 10.0 đến 40.0 độ C và chênh lệch so với giá trị cũ lớn hơn 0.1, giúp chống lại các tín hiệu nhiễu và lệnh điều khiển sai lệch.

3.5.4. Giải pháp bảo mật và định tuyến qua Mesh VPN

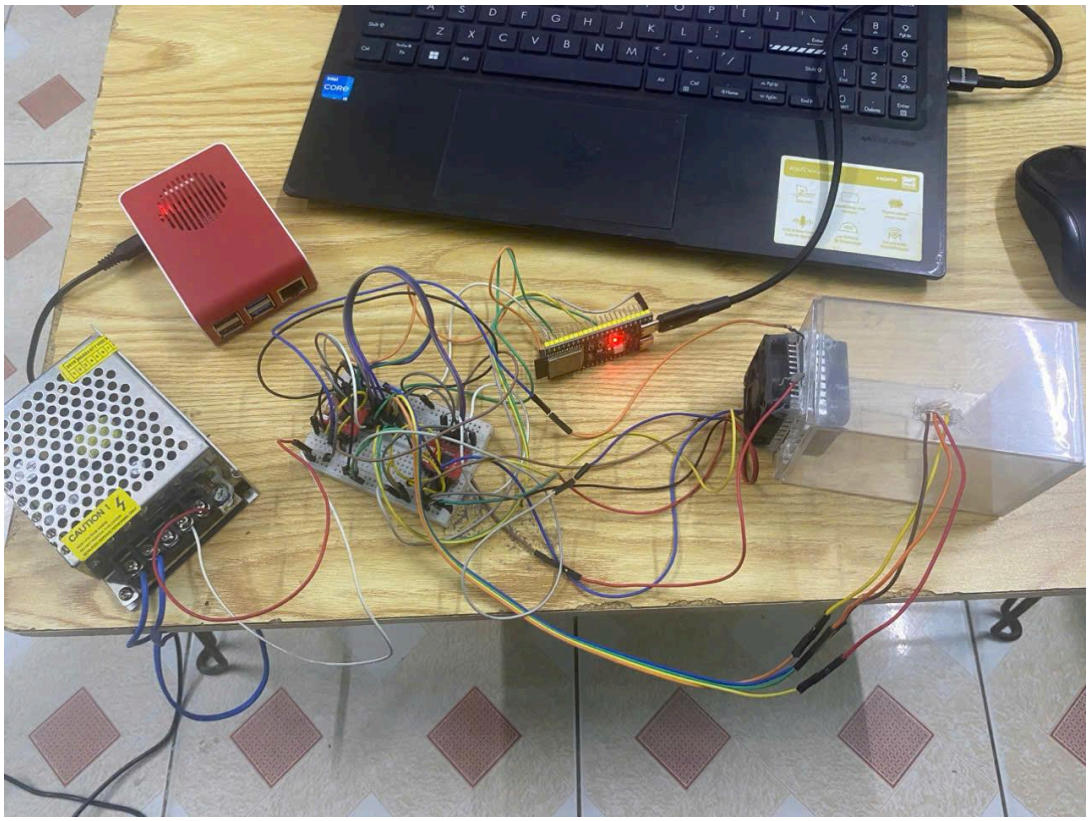
- Để điều khiển thiết bị nằm ở mạng nội bộ 172.20.10.x từ bất kỳ đâu qua Internet, hệ thống tích hợp giải pháp mạng ảo dạng lưới (Mesh VPN) thông qua Tailscale trên Raspberry Pi. Các gói tin Web và MQTT sẽ được mã hóa đầu-cuối (End-to-End Encryption) theo chuẩn WireGuard và đi thẳng trực tiếp (Peer-to-Peer) qua các tầng bảo mật NAT của Router nhà mạng mà không cần thao tác Port Forwarding rủi ro.

Chương 4. KẾT QUẢ THỰC NGHIỆM

Link video demo: Temp_Control.mp4

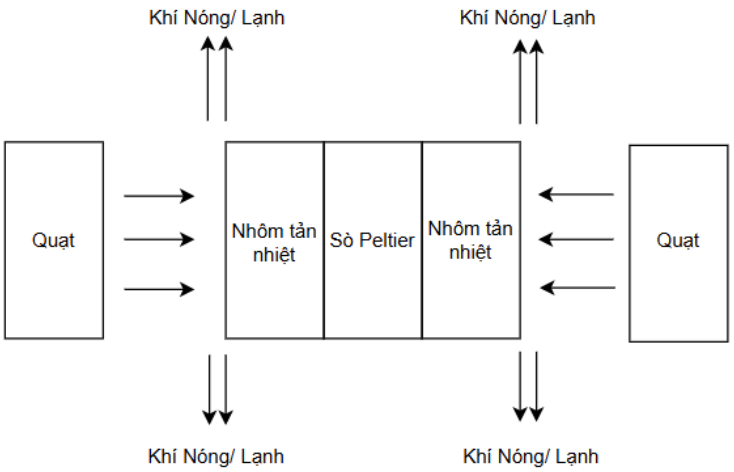
4.1. Kết quả thi công phần cứng

Nhóm đã thiết kế thành công được mô hình nhỏ, điều chỉnh nhiệt độ trong chiếc hộp nhỏ mô phỏng thay cho việc điều chỉnh nhiệt độ cho không gian phòng lớn hơn.



Hình 1.7: Kết quả lắp ghép phần cứng

Khối chấp hành chính ở đây sẽ là một hệ thống sò nóng lạnh kết hợp với quạt và khối nhôm tản nhiệt được kẹp vào nhau theo sơ đồ sau:



Hình 1.8: Sơ đồ nguyên lí hoạt động khối chấp hành

Khí Nóng/ Lạnh từ sò sẽ được hấp thụ bởi các tấm nhôm tản nhiệt, sau đó quạt sẽ thổi vào tấm nhôm và đẩy khí Nóng/Lạnh này qua các khe rồi ra ngoài môi trường.

Từ đó khi cấp nguồn mặt bên trong hộp sẽ thổi ra khí nóng/lạnh ra nhiệt độ trong hộp. Cảm biến SHT31 được treo lơ lửng trong hộp sẽ nhận được ngay luồng khí này và gửi dữ liệu đo được cho ESP32-S3.

4.2. Kết quả truyền thông mạng và giao diện người dùng

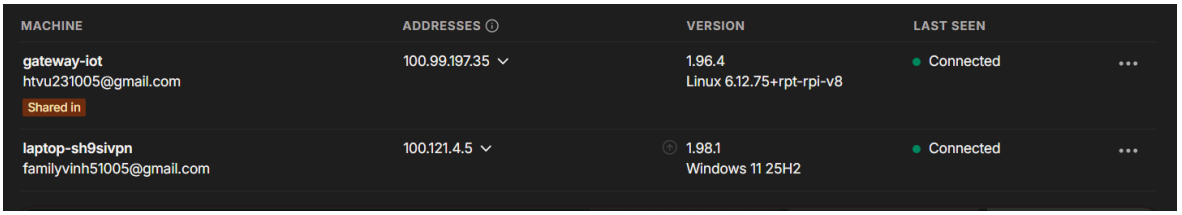
4.2.1. Kết quả khởi tạo kết nối và truyền tin tại thiết bị biên.



- **Kết nối Wi-Fi:** Giao diện mạng example_netif_sta đã xin cấp phát thành công địa chỉ IPv4 nội bộ là 172.20.10.13 từ Router.
- **Khởi tạo MQTT:** Cửa sổ log ghi nhận dòng thông báo "Đã khởi động xong mạng và MQTT!", chứng tỏ thiết bị đã bắt tay thành công với MQTT Broker để sẵn sàng cho mô hình Publish/Subscribe.

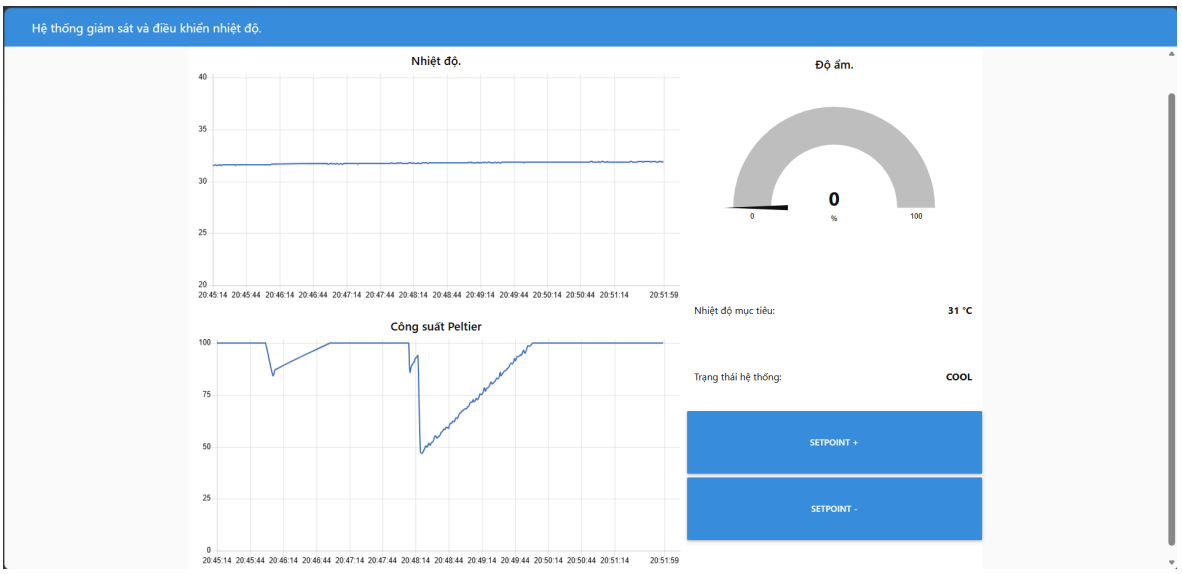
- **Truyền tải bản tin (Publish):** Ngay sau khi kết nối, hệ thống lập tức đóng gói và đẩy bản tin giám sát đầu tiên lên topic. Dữ liệu ghi nhận rõ các thông số: Nhiệt độ hiện tại (31.67°C), Nhiệt độ mục tiêu (30.0°C) và mức băm xung PWM hiện tại (4095 - công suất tối đa), chứng minh luồng dữ liệu đẩy từ trạm đo lường lên hệ thống đám mây hoạt động ổn định, không bị nghẽn hay suy hao gói tin.

4.2.2. Kết quả xuyên thấu tường lửa bằng mạng riêng ảo Tailscale.



- Hệ thống ghi nhận trạng thái Connected của cả 2 thiết bị cốt lõi trong mô hình mạng lưới (Mesh VPN): một Node đóng vai trò là gateway-iot (chạy Linux, đóng vai trò trạm trung chuyển/Broker) và một thiết bị người dùng cuối là laptop-sh9sivpn (chạy Windows 11).
- Mỗi thiết bị đã được cấp phát một dải IP ảo an toàn (lần lượt là 100.99.197.35 và 100.121.4.5). Nhờ hạ tầng này, người quản trị từ Laptop sử dụng bất kỳ mạng Internet nào (như 4G hay Wi-Fi công cộng) cũng có thể truy cập thẳng vào Gateway IoT nội bộ để điều khiển hệ thống mà không cần thực hiện mở cổng (Port Forwarding) kém bảo mật trên Router vật lý.

4.2.3. Kết quả thiết kế giao diện giám sát và điều khiển.



Hình 1.9: Giao diện giám sát và điều khiển

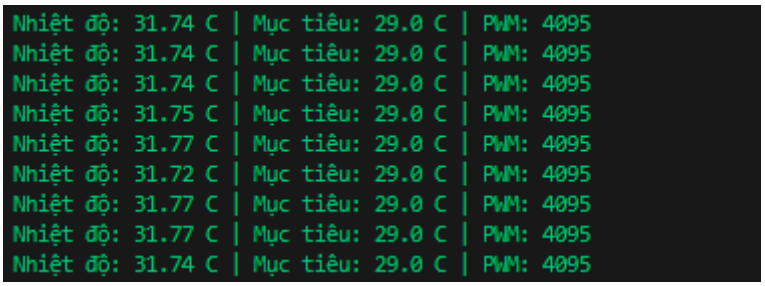
Hình ảnh Dashboard thể hiện tính tương tác hai chiều (Giám sát - Điều khiển) của hệ thống IoT(Headless Device):

- Khu vực Giám sát thời gian thực: Giao diện hiển thị trực quan các biểu đồ dạng đường (Line chart) ghi nhận lịch sử biến thiên của hệ thống.
    - Biểu đồ Nhiệt độ cho thấy hệ thống đang duy trì môi trường khá ổn định quanh mốc 31°C - 32°C.
    - Biểu đồ Công suất Peltier minh họa rõ ràng hoạt động của thuật toán điều khiển PID khi liên tục điều chỉnh mức PWM (giảm sâu rồi tăng dần lại lên 100%) để cố gắng bám sát Setpoint.
- Khu vực hiển thị trạng thái: Các thông số tĩnh được cập nhật liên tục như Trạng thái hệ thống đang ở chế độ làm mát (COOL) và Nhiệt độ mục tiêu hiện tại đang được đặt là 31°C.
- Khu vực tương tác (Điều khiển từ xa): Cụm nút nhấn SETPOINT + và SETPOINT - đóng vai trò là cơ cấu điều khiển phản hồi thay thế cho nút nhấn cơ học. Khi người dùng thao tác trên giao diện này, tín hiệu sẽ lập tức

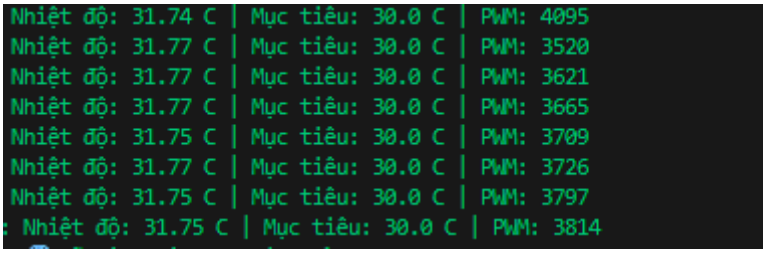
được Publish xuống trạm vi điều khiển thông qua MQTT, thực hiện thay đổi giá trị hoạt động ngay tức thì.

4.3. Đánh giá đáp ứng của thuật toán PID

Nhiệt độ hiện tại đang là 31.74°C, setpoint ở mức 29°C tức Error lúc này đang là -2.74 nằm ngoài khoảng [-2;2] nên PWM sẽ ở mức cao nhất 4095.



Chỉnh setpoint lên ở mức 30°C -> Error = -1.74 nằm trong khoảng [-2;2] nên PID sẽ bắt đầu tính giá trị PWM để xuất ra sò. Có thể thấy khi nhiệt độ dao động nhỏ quanh mức cũ lâu không thay đổi nhiều thì sai số tích lũy (Tích phân I của PID) sẽ tăng dần và điều chỉnh PWM càng lên cao.



Tiếp tục chỉnh Setpoint lên mức 31°C thì nhận thấy nhiệt độ hiện tại đang dần tiến đến setpoint nên PWM cũng sẽ giảm dần về mức 2000 (50% công suất) và bắt đầu tích lũy dần sai số khi thấy nhiệt độ có khả năng chưa chạm tới setpoint theo thời gian.

```
Nhiệt độ: 31.78 C | Mục tiêu: 31.0 C | PWM: 2859
Nhiệt độ: 31.75 C | Mục tiêu: 31.0 C | PWM: 1965
Nhiệt độ: 31.74 C | Mục tiêu: 31.0 C | PWM: 1927
Nhiệt độ: 31.75 C | Mục tiêu: 31.0 C | PWM: 1919
Nhiệt độ: 31.77 C | Mục tiêu: 31.0 C | PWM: 1964
Nhiệt độ: 31.78 C | Mục tiêu: 31.0 C | PWM: 2010
Nhiệt độ: 31.77 C | Mục tiêu: 31.0 C | PWM: 2062
Nhiệt độ: 31.77 C | Mục tiêu: 31.0 C | PWM: 2049
Nhiệt độ: 31.78 C | Mục tiêu: 31.0 C | PWM: 2068
Nhiệt độ: 31.75 C | Mục tiêu: 31.0 C | PWM: 2120
```

⇒ **Kết luận:** Có thể thấy thuật toán PID xuất ra xung PWM đã hoạt động chính xác cho hệ thống điều khiển.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Sau thời gian nghiên cứu, tìm hiểu nhóm đã thiết kế được hệ thống điều chỉnh nhiệt độ thông minh cho không gian nhỏ, cụ thể nhóm đã thực hiện mô phỏng trên hộp kín nhỏ, tạo tiền đề để phát triển lên cho môi trường lớn sau này như phòng ốc, nhà kính,... Thông qua việc sử dụng thuật toán PID để khiến cho việc điều chỉnh diễn ra một cách tự động, thêm việc thiết kế một giao diện người dùng trên web để có thể tương tác với hệ thống từ xa giúp cho việc điều chỉnh nhiệt độ mong muốn thêm phần dễ dàng.

5.2. Phần hạn chế của đề tài

- Tuy nhiên trong quá trình thực hiện nhóm có gặp không ít khó khăn do hệ thống sử dụng khối chấp hành chính cho việc điều chỉnh nhiệt độ còn qúa thấp về mặt công suất nên việc điều chỉnh diễn ra khá chậm và lâu.
- Hệ thống chưa tích hợp thêm việc theo dõi và điều chỉnh độ ẩm do khó khăn trong việc tìm mua các linh kiện có thể đáp ứng phù hợp cho đề tài.

5.3. Hướng phát triển

- Tích hợp thêm việc theo dõi và điều chỉnh độ ẩm cho phòng.
- Sử dụng các linh kiện công suất cao hơn để hệ thống hoạt động một cách nhanh chóng và thuận lợi.
- Ứng dụng bộ điều khiển khác như Logic mờ thay thế cho các phương pháp điều chỉnh PID.
- Phát triển một mobile app riêng biệt.